

*#Lab 7 : BN vs Dropout*

*#Use the CIFAR dataset to build a deep neural network model and analyze the effect of batch normalization and dropout*

```
import torch,torch.nn as nn,torchvision
from torchvision import transforms as T
```

```
t=T.ToTensor()
```

```
tr=torchvision.datasets.CIFAR10('./data',train=True,download=True,transform=t)
```

```
te=torchvision.datasets.CIFAR10('./data',train=False,download=True,transform=t)
```

```
trl=torch.utils.data.DataLoader(torch.utils.data.Subset(tr,range(10000)),64,1)
```

```
tel=torch.utils.data.DataLoader(torch.utils.data.Subset(te,range(2000)),64)
```

*# CNN with BatchNorm + Dropout*

```
def Net():
```

```
    return nn.Sequential(
        nn.Conv2d(3,32,3,padding=1),
        nn.BatchNorm2d(32),
        nn.ReLU(),
        nn.MaxPool2d(2),

        nn.Dropout(0.3),

        nn.Flatten(),
        nn.Linear(32*16*16,128),
        nn.ReLU(),

        nn.Dropout(0.5),

        nn.Linear(128,10)
    )
```

```
m=Net()
```

```
o=torch.optim.Adam(m.parameters(),1e-3)
```

```
c=nn.CrossEntropyLoss()
```

```
for e in range(3):
```

```
    for x,y in trl:
        o.zero_grad()
        l=c(m(x),y)
        l.backward()
        o.step()
```

```
    print(f'Epoch {e+1} Loss:',l.item())
```

cor=tot=0

```
with torch.no_grad():
    for x,y in tel:
        p=m(x).argmax(1)
        cor+=(p==y).sum().item()
        tot+=y.size(0)

print('Accuracy:',100*cor/tot,'%')
```

```
Downloading https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz to
./data\cifar-10-python.tar.gz
```

```
100%|██████████████████████████████████████████████████████████████|  
170498071/170498071 [08:16<00:00, 343448.33it/s]
```

```
Extracting ./data\cifar-10-python.tar.gz to ./data
Files already downloaded and verified
Epoch 1 Loss: 1.9746931791305542
Epoch 2 Loss: 2.3700921535491943
Epoch 3 Loss: 1.424789309501648
Accuracy: 35.55 %
```

## #Lab 8 : Optimizer Comparison

#Implement a simple Convolutional Neural Network (CNN) and compare the performance of any two optimizers on the given image dataset

```
import torch,torch.nn as nn,torchvision,matplotlib.pyplot as plt
from torchvision import transforms as T
```

```
t=T.ToTensor()
```

```
d=torchvision.datasets.MNIST('./data',train=True,download=True,transform=t)
l=torch.utils.data.DataLoader(torch.utils.data.Subset(d,range(10000)),
256,1)
```

## # Model

```
def net():
    return nn.Sequential(
        nn.Conv2d(1, 16, 3), nn.ReLU(), nn.MaxPool2d(2),
        nn.Flatten(), nn.Linear(2704, 10)
    )
```

```
m1,m2=net(),net()
```

```
o1=torch.optim.Adam(m1.parameters(),1e-3)
o2=torch.optim.SGD(m2.parameters(),1e-2)
```

```
c=nn.CrossEntropyLoss()
```

```

a,s=[],[]
for _ in range(3):
    # Adam
    for x,y in l:
        o1.zero_grad()
        loss1=c(m1(x),y)
        loss1.backward()
        o1.step()

    # SGD
    for x,y in l:
        o2.zero_grad()
        loss2=c(m2(x),y)
        loss2.backward()
        o2.step()

    a.append(loss1.item())
    s.append(loss2.item())

print("Adam Loss:",a[-1])
print("SGD Loss:",s[-1])

```

```

plt.plot(a,label='Adam')
plt.plot(s,label='SGD')
plt.legend()
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.show()

```

Downloading <http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz>  
 Failed to download (trying next):  
 HTTP Error 404: Not Found

Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/train-images-idx3-ubyte.gz>  
 Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/train-images-idx3-ubyte.gz> to ./data\MNIST\raw\train-images-idx3-ubyte.gz

100%|

9912422/9912422 [02:40<00:00, 61823.38it/s]

Extracting ./data\MNIST\raw\train-images-idx3-ubyte.gz to ./data\MNIST\raw

Downloading <http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz>

Failed to download (trying next):  
HTTP Error 404: Not Found

Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/train-labels-idx1-ubyte.gz>  
Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/train-labels-idx1-ubyte.gz> to ./data\MNIST\raw\train-labels-idx1-ubyte.gz

100%|

| 28881/28881 [00:00<00:00, 136795.19it/s]

Extracting ./data\MNIST\raw\train-labels-idx1-ubyte.gz to ./data\MNIST\raw

Downloading <http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz>  
Failed to download (trying next):  
HTTP Error 404: Not Found

Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz>  
Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz> to ./data\MNIST\raw\t10k-images-idx3-ubyte.gz

100%|

| 1648877/1648877 [00:03<00:00, 485158.75it/s]

Extracting ./data\MNIST\raw\t10k-images-idx3-ubyte.gz to ./data\MNIST\raw

Downloading <http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz>  
Failed to download (trying next):  
HTTP Error 404: Not Found

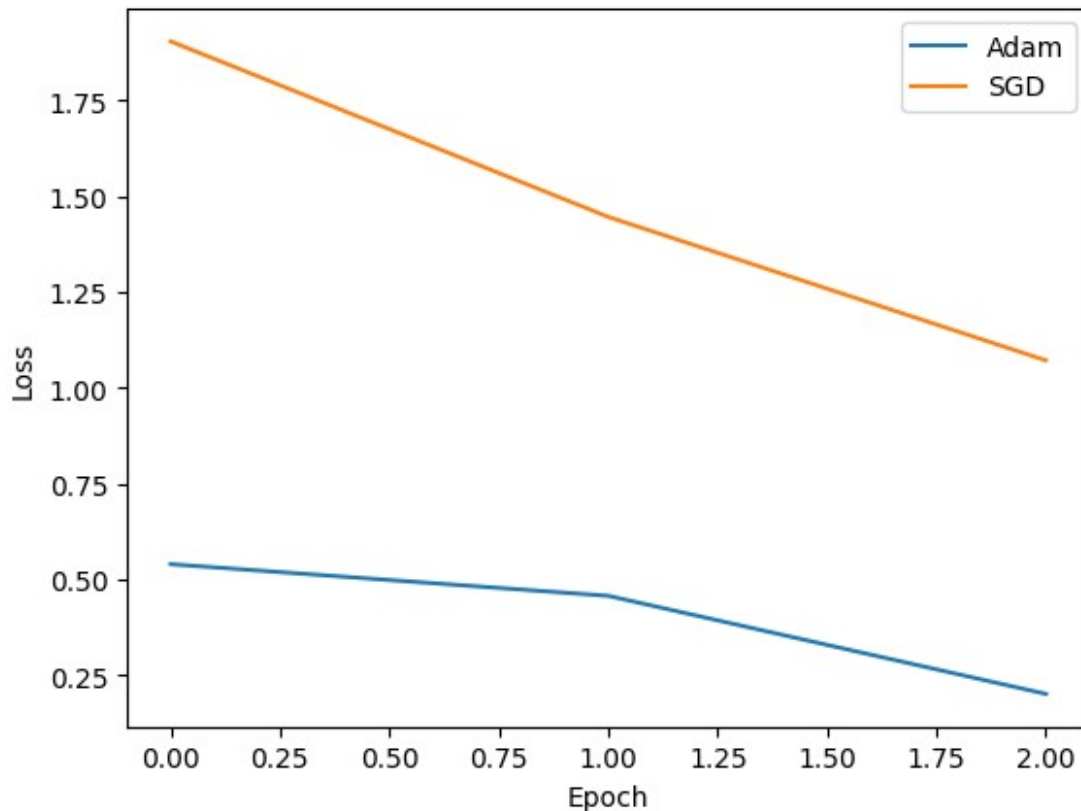
Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz>  
Downloading <https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz> to ./data\MNIST\raw\t10k-labels-idx1-ubyte.gz

100%|

| 4542/4542 [00:00<00:00, 2271977.19it/s]

Extracting ./data\MNIST\raw\t10k-labels-idx1-ubyte.gz to ./data\MNIST\raw

Adam Loss: 0.20193515717983246  
SGD Loss: 1.0714077949523926



```
!pip install torch==2.2.2 torchvision==0.17.2 torchaudio==2.2.2
```

```
Collecting torch==2.2.2
```

```
Using cached torch-2.2.2-cp311-cp311-win_amd64.whl.metadata (26 kB)
```

```
Collecting torchvision==0.17.2
```

```
Using cached torchvision-0.17.2-cp311-cp311-win_amd64.whl.metadata (6.6 kB)
```

```
Collecting torchaudio==2.2.2
```

```
Using cached torchaudio-2.2.2-cp311-cp311-win_amd64.whl.metadata (6.4 kB)
```

```
Requirement already satisfied: filelock in C:\Program Files\Python311\Lib\site-packages (from torch==2.2.2) (3.20.0)
```

```
Requirement already satisfied: typing-extensions>=4.8.0 in C:\Program Files\Python311\Lib\site-packages (from torch==2.2.2) (4.15.0)
```

```
Requirement already satisfied: sympy in C:\Program Files\Python311\Lib\site-packages (from torch==2.2.2) (1.14.0)
```

```
Requirement already satisfied: networkx in C:\Program Files\Python311\Lib\site-packages (from torch==2.2.2) (3.5)
```

```
Requirement already satisfied: jinja2 in C:\Program Files\Python311\Lib\site-packages (from torch==2.2.2) (3.1.6)
```

```
Requirement already satisfied: fsspec in C:\Program Files\Python311\Lib\site-packages (from torch==2.2.2) (2025.9.0)
```

```
Requirement already satisfied: numpy in C:\Program Files\Python311\Lib\site-packages (from torchvision==0.17.2) (1.26.4)
```



[illegible]

[illegible]



[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]



[illegible]



[illegible]

Successfully installed torch-2.2.2 torchaudio-2.2.2 torchvision-0.17.2

```
ERROR: pip's dependency resolver does not currently take into account
all the packages that are installed. This behaviour is the source of
the following dependency conflicts.
```

```
torch-directml 0.2.5.dev240914 requires torch==2.4.1, but you have
torch 2.2.2 which is incompatible.
```

```
torch-directml 0.2.5.dev240914 requires torchvision==0.19.1, but you
have torchvision 0.17.2 which is incompatible.
```

```
xformers 0.0.33.post1 requires torch==2.9.0, but you have torch 2.2.2
```

which is incompatible.

```
[notice] A new release of pip is available: 26.0.1 -> 26.1.1  
[notice] To update, run: python.exe -m pip install --upgrade pip
```

*#LAB - 09*

*#Implement LeNet-5 architecture for image classification on MNIST dataset. Train the network*

*#and analyze classification performance on image data.*

```
import torch,torch.nn as nn,torchvision,matplotlib.pyplot as plt  
from torchvision import transforms as T  
  
t=T.Compose([T.Resize((32,32)),T.ToTensor()])  
  
tr=torchvision.datasets.MNIST('./data',train=True,download=True,transform=t)  
te=torchvision.datasets.MNIST('./data',train=False,download=True,transform=t)  
  
trl=torch.utils.data.DataLoader(torch.utils.data.Subset(tr,range(10000)),64,1)  
tel=torch.utils.data.DataLoader(torch.utils.data.Subset(te,range(2000)),64)  
  
# LeNet-5  
m=nn.Sequential(  
    nn.Conv2d(1,6,5),nn.Tanh(),nn.AvgPool2d(2),  
    nn.Conv2d(6,16,5),nn.Tanh(),nn.AvgPool2d(2),  
    nn.Flatten(),  
    nn.Linear(400,120),nn.Tanh(),  
    nn.Linear(120,84),nn.Tanh(),  
    nn.Linear(84,10)  
)  
  
o=torch.optim.Adam(m.parameters(),1e-3)  
c=nn.CrossEntropyLoss()  
  
losses=[]  
  
for e in range(5):  
    for x,y in trl:  
        o.zero_grad()  
        l=c(m(x),y)  
        l.backward()  
        o.step()  
  
        losses.append(l.item())  
        print(f'Epoch {e+1} Loss:',l.item())
```

*# Accuracy*

```

cor=tot=0

with torch.no_grad():
    for x,y in tel:
        p=m(x).argmax(1)
        cor+=(p==y).sum().item()
        tot+=y.size(0)

print("Accuracy:",100*cor/tot,"%")

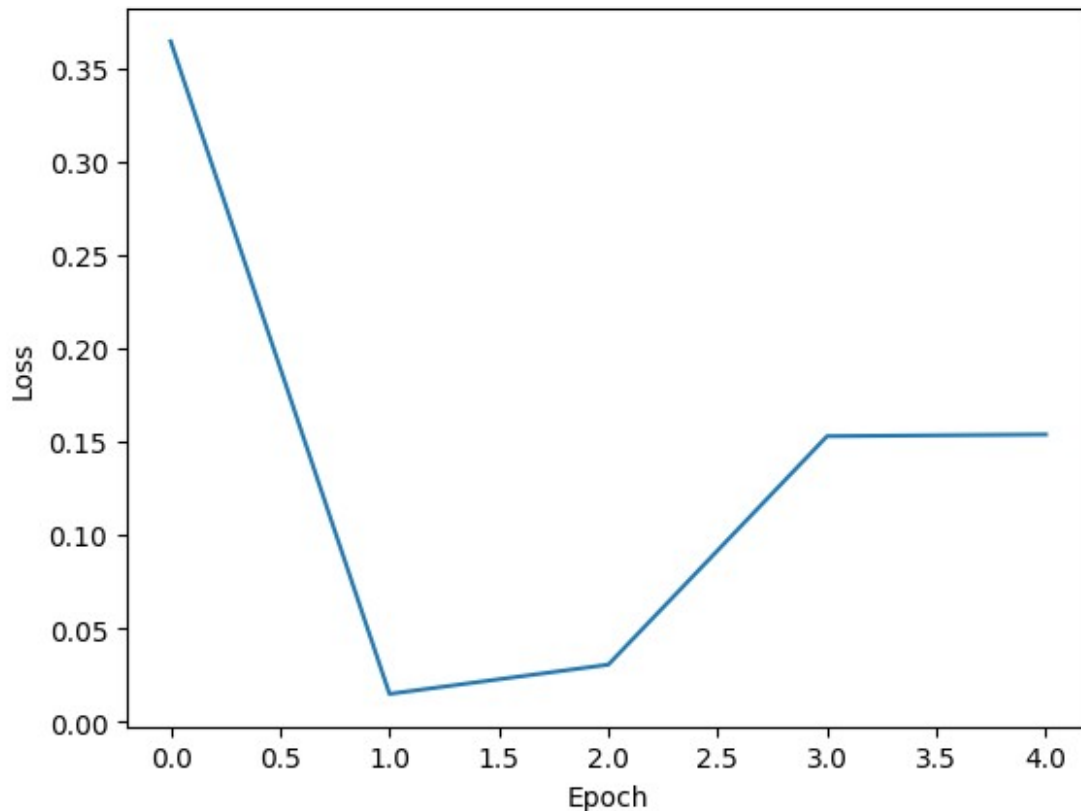
# Graph
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.show()

# Prediction
# x,y=te[0]
# p=m(x.unsqueeze(0)).argmax(1).item()

# plt.imshow(x.squeeze(),cmap='gray')
# plt.title(f'A:{y} P:{p}')
# plt.axis('off')
# plt.show()

Epoch 1 Loss: 0.3643829822540283
Epoch 2 Loss: 0.014839737676084042
Epoch 3 Loss: 0.030497262254357338
Epoch 4 Loss: 0.15289056301116943
Epoch 5 Loss: 0.1537809520959854
Accuracy: 94.2 %

```



```
#10 - Implement the AlexNet architecture using a deep learning
framework and apply it to the given
#image classification task
import torch,requests,pandas as pd
import torch.nn.functional as F
from torchvision import models,transforms as T
from PIL import Image
import plotly.express as px

# AlexNet
m=models.alexnet(pretrained=True)
m.eval()

# Image
img=Image.open('g1.jpg')

t=T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])
])

x=t(img).unsqueeze(0)
```

```

# Prediction
with torch.no_grad():
    p=F.softmax(m(x),1)[0]

# Labels
lab=requests.get(

'https://raw.githubusercontent.com/pytorch/hub/master/imagenet_classes
.txt'
).text.splitlines()

# Top 5
v,i=torch.topk(p,5)

df=pd.DataFrame({
    'Class':[lab[j] for j in i],
    'Prob':v.numpy()
})

print(df)

# Graph
fig=px.bar(df,x='Prob',y='Class',orientation='h',
           title='AlexNet Top 5 Predictions',
           color='Prob')

fig.show()

# 11 - Demonstrate the concept of language modeling using RNN, which
involves training a model to predict the
#next word in a sequence of words.

import torch,torch.nn as nn,torch.optim as optim

d='cuda' if torch.cuda.is_available() else 'cpu'

txt='pytorch is great for building deep learning models and rnn models
are fun'.split()

v=sorted(set(txt))
w2i={w:i for i,w in enumerate(v)}
i2w={i:w for w,i in w2i.items()}

X=torch.tensor([w2i[w] for w in txt[:-1]]).unsqueeze(1).to(d)
Y=torch.tensor([w2i[w] for w in txt[1:]]).to(d)

class RNN(nn.Module):
    def __init__(s):
        super().__init__()
        s.e=nn.Embedding(len(v),16)
        s.r=nn.RNN(16,32,batch_first=True)

```

```

        s.f=nn.Linear(32,len(v))

    def forward(s,x):
        o,_=s.r(s.e(x))
        return s.f(o.squeeze(1))

m=RNN().to(d)

o=optim.Adam(m.parameters(),0.05)
c=nn.CrossEntropyLoss()

# Train
for e in range(100):
    o.zero_grad()

    l=c(m(X),Y)

    l.backward()
    o.step()

    if (e+1)%25==0:
        print(f'Epoch {e+1} Loss:',l.item())

# Predict
def p(w):
    x=torch.tensor([[w2i[w]]]).to(d)
    return i2w[m(x).argmax(1).item()]

print("\nAfter 'pytorch' -->",p('pytorch'))
print("After 'building' -->",p('building'))

```

```

Epoch 25 Loss: 0.11556621640920639
Epoch 50 Loss: 0.11568835377693176
Epoch 75 Loss: 0.11556496471166611
Epoch 100 Loss: 0.11560431122779846

```

```

After 'pytorch' --> is
After 'building' --> deep

```

*# 12 - Implementan LSTM-based text generation model and train the model to generate text sequences.*

```

import torch,torch.nn as nn,torch.optim as optim

d='cuda' if torch.cuda.is_available() else 'cpu'

txt='pytorch is great for building deep learning models and rnn models
are fun'.split()

v=sorted(set(txt))
w2i={w:i for i,w in enumerate(v)}

```

```

i2w={i:w for w,i in w2i.items()}

X=torch.tensor([w2i[w] for w in txt[:-1]]).unsqueeze(1).to(d)
Y=torch.tensor([w2i[w] for w in txt[1:]]).to(d)

class model(nn.Module):
    def __init__(s):
        super().__init__()
        s.e=nn.Embedding(len(v),16)
        s.r=nn.LSTM(16,32,batch_first=True)
        s.f=nn.Linear(32,len(v))

    def forward(s,x):
        o,_=s.r(s.e(x))
        return s.f(o.squeeze(1))

m=model().to(d)

o=optim.Adam(m.parameters(),0.05)
c=nn.CrossEntropyLoss()

# Train
for e in range(100):
    o.zero_grad()

    l=c(m(X),Y)

    l.backward()
    o.step()

    if (e+1)%25==0:
        print(f'Epoch {e+1} Loss:',l.item())

# Predict
def p(w):
    x=torch.tensor([[w2i[w]]]).to(d)
    return i2w[m(x).argmax(1).item()]

w1='for'
w2='rnn'
print(f"\nAfter {w1} -->",p(w1))
print(f"After {w2} -->",p(w2))

Epoch 25 Loss: 0.11573917418718338
Epoch 50 Loss: 0.11556198447942734
Epoch 75 Loss: 0.11555219441652298
Epoch 100 Loss: 0.11554928869009018

After for --> building
After rnn --> models

```